CS780 Capstone Project: OBELIX - The Warehouse Robot

Om Mallick
251110055.
ommallick25@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

This work addresses the OBELIX warehouse robot challenge: training an agent to find, attach, and push a box to arena boundaries under partial observability. We systematically evaluated multiple RL algorithms across three difficulty levels (static, blinking, moving). Our key finding is that temporal memory mechanisms—state stacking and LSTM—are crucial for handling non-stationary box dynamics, with simple concatenation matching recurrent performance. Our final solution, **VPG with State Stacking**, achieved top performance (≈–1827.71) by combining a learned value baseline with explicit four-frame temporal context.

1 Competition Result

Codabench Username: 0_251110055
Final leaderboard rank (Test Phase): 38
Leaderboard rank – Level 1 (static): 72
Leaderboard rank – Level 2 (blinking): 60
Leaderboard rank – Level 3 (moving + blinking): 17

Number of valid submissions made:

Table 1: Submission count across competition phases

Phase	**Count**	**Algorithms/Approaches Tested**
Level 1 (Static)	3	Random baseline, Heuristic controller, Pretrained weights
Level 2 (Blinking)	17	Random, DDQN Variants, Q-Learning + State Clustering, Rotate, Heuristic, REINFORCE, A2C, A3C, PPO, DYNA-Q, Braitenberg Vehicle, Noisy DDQN Variants
Level 3 (Moving)	11	Random, Rotate, Heuristic, Braitenberg Vehicle, DDQN and Noisy DDQN Variants
Test Phase (Final)	50	Comprehensive evaluation including: Value-based variants, Policy gradients, Actor-critic, Model-based, and advanced combinations
Total Submissions	**81**	Across all development and test phases

2 Introduction and Problem Description

Autonomous warehouse robotics represents a critical application domain for reinforcement learning (RL), where robots must perform complex manipulation tasks in dynamic, partially observable environments. This work addresses the OBELIX challenge, a simulated warehouse navigation task inspired by the seminal 1991 work of Mahadevan and Connell [1] and built upon modern RL foundations [2].

Task Description. The core objective requires controlling a mobile robot to locate a grey box, attach to it, and push it to a designated boundary. This sequence decomposes into four conceptual behaviors: *Find* (navigate using sensors), *Attach* (make contact), *Push* (move to boundaries), and *Unwedge* (recover from wall collisions). Episodes terminate successfully when the box touches the arena boundary (+2000 reward) or unsuccessfully at the maximum step horizon.

Environment and Observation. The robot operates in a 500×500 pixel arena with discrete 18-bit observations: 16 sonar bits (near/far ranges), 1 infrared contact sensor, and 1 stuck-flag. The action space comprises five discrete movements: rotate left 45°, rotate left 22.5°, move forward, rotate right 22.5°, and rotate right 45°. Rewards include one-time sensor bonuses (+1 to +5), per-step penalty (-1), stuck penalty (-200), and success bonus (+2000).

Difficulty Levels. The competition progresses through three tiers: **Level 1** (static box), **Level 2** (blinking box: 30–60 steps visible, 10–30 invisible), and **Level 3** (moving + blinking: 2 pixels/step velocity). An optional wall obstacle introduces a vertical barrier with a narrow opening.

Challenges: A POMDP Setting. This environment constitutes a Partially Observable Markov Decision Process (POMDP). The robot lacks absolute position coordinates and must infer spatial relationships from local 18-bit sensor readings. Non-stationary box dynamics in Levels 2–3 violate the Markov property, necessitating memory mechanisms. The sparse reward structure and -200 stuck penalty create challenging exploration dynamics.

Objective. Our goal is to develop robust RL policies maximizing mean cumulative reward across all difficulty levels while generalizing to wall configurations. This requires balancing exploration, exploitation, and recovery under severe perceptual limitations.

3 Approach and Model Evolution

3.1 Baseline and Initial Attempts

Phase 1 (Level 1 – Static Box): The Random policy and Heuristic controller both performed poorly, confirming that undirected exploration and naive hand-crafted behaviors fail to handle the sparse reward structure and precise alignment required for box attachment.

3.2 Evolution Across Phases

Phase 2 (Level 2 – Blinking Box): Value-Based Foundations We began with **Double DQN (DDQN)**, which learned sensor activation but struggled with long-horizon credit assignment for complete task sequences. **D3QN** and **D3QN-PER** improved value estimation through better state-value separation and prioritized sampling. We also implemented **Q-Learning with State Clustering** inspired by Mahadevan & Connell's original work.

Policy Gradient Exploration: REINFORCE, **A2C**, **A3C**, and **PPO** exhibited high variance in Phase 2 due to sparse rewards, revealing that value-based methods offered more stable learning.

Exploration Improvements: Noisy DDQN and **Noisy D3QN** replaced ϵ-greedy with parameter space noise, providing adaptive exploration that decayed naturally during training.

Classical Methods: We evaluated **DYNA-Q** and **Braitenberg Vehicle** approaches for comparison.

Phase 3 (Level 3 – Moving + Blinking): Baseline Evaluation We systematically evaluated Phase 2 architectures (DDQN, D3QN, D3QN-PER, Noisy DDQN, Noisy D3QN) against moving boxes to establish rigorous baselines. Results confirmed that partial observability degrades feedforward policy performance, justifying the recurrent and state-stacking architectures implemented in the Final Phase.

3.3 Test Phase: Memory and Architecture Ablations

- **State Stacking:** Concatenation of last 4 observations (72-dimensional input) for DDQN+State Stacking, VPG+State Stacking, and D3QN+State Stacking, providing temporal context without recurrence.
- **Recurrent Architectures:** DDQN and Noisy DDQN LSTM variants processing observation sequences to infer velocity and predict positions during blinking.
- **Policy Gradient Methods:** REINFORCE, VPG, and GAE, achieving competitive performance with temporal context.
- **Actor-Critic and Continuous Methods:** A2C, A3C, DDPG, TD3, SAC, and PPO. DDPG and TD3 faced challenges with discrete action spaces.
- **Model-Based Methods:** MCTS and DYNA-Q exhibited high computational overhead.

Best Performers: Submission 26 (VPG + State Stacking), **Submission 14 (Noisy DDQN + LSTM)**, and **Submission 29 (GAE)** achieved approximately -1827 to -1828.

3.4 Detailed Description of Final Model

We present **VPG with State Stacking** as our final solution, combining Vanilla Policy Gradient with a learned value baseline and explicit four-frame temporal context via observation concatenation.

State Representation: The model processes a 72-dimensional stacked observation formed by concatenating the current and three most recent 18-bit frames. This gives the policy explicit access to recent sensor history, enabling it to infer box motion and maintain a short-term trajectory estimate without the training instability of recurrent networks.

Network Architecture: Figure 1 illustrates the architecture: the stacked 72-dimensional input is processed by separate actor and critic networks, each with two hidden layers of 128 units, producing action probabilities and a scalar state value, respectively.

- **Actor:** Linear(72→128) → ReLU → Linear(128→128) → ReLU → Linear(128→5) → Softmax
- **Critic:** Linear(72→128) → ReLU → Linear(128→128) → ReLU → Linear(128→1)

Exploration Strategy: Stochastic policy sampling provides natural exploration during training. Entropy in the categorical distribution encourages the actor to avoid premature convergence. At evaluation time, the policy selects the greedy (argmax) action.

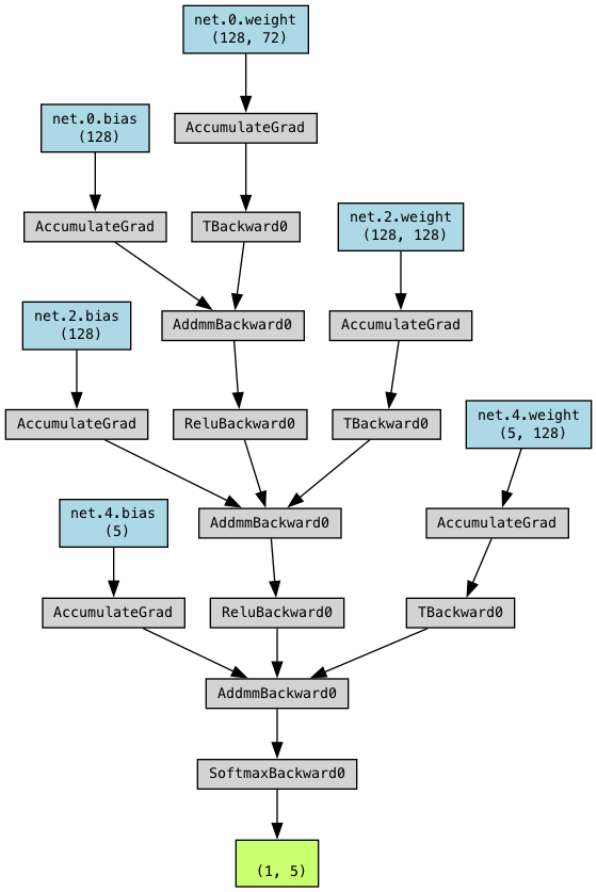
Advantage Estimation: Monte Carlo returns $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$ serve as targets for both the critic loss and the policy gradient. Advantages $A_t = G_t - V(s_t)$ are normalized per episode to stabilise gradient magnitudes:

$$\hat{A}_t = \frac{A_t - \mu_A}{\sigma_A + \epsilon}$$

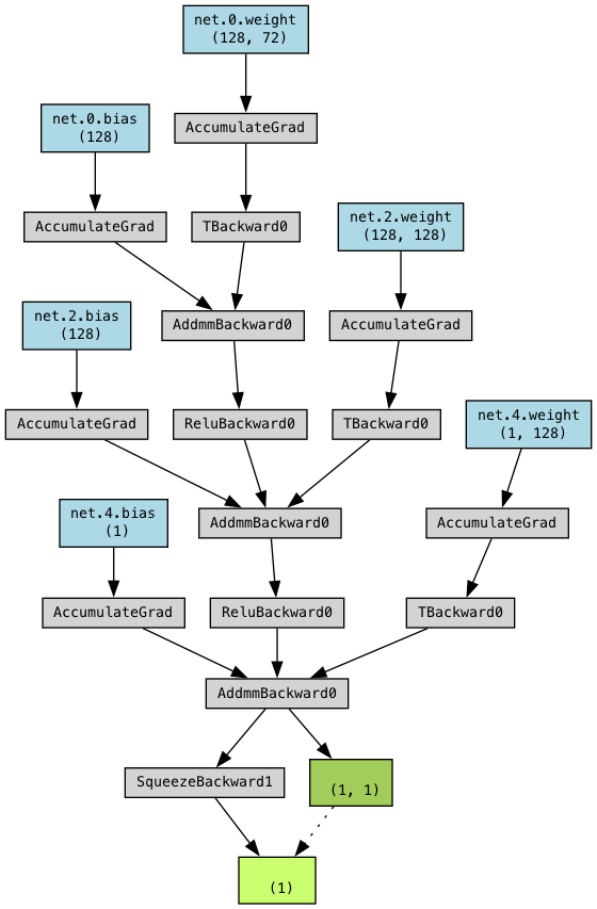
Training Details:

- Actor optimizer: Adam (3×10^{-4})
- Critic optimizer: Adam (10^{-3})
- $\gamma = 0.99$, gradient clipping norm 1.0

- Stack size: 4 frames (72-dimensional input)
- Model selection by best 100-episode moving average of training return



(a) VPG Actor Architecture



(b) VPG Critic Architecture

Figure 1: Overall description comparing the two VPG architectures: stacked 72-dim observations feed separate actor and critic MLP heads.

4 Experiments and Implementation Details

4.1 Code and Reproducibility

Code Repository: `https://github.com/ommallick02/CS780-OBELIX`
Demonstration Video: `https://youtu.be/LxSVDIQkA_A`

All source code is available in the repository linked above, including agent implementations, training scripts, pre-trained weights, and evaluation tools. The video demonstration of the project is linked above as well.

4.2 Training Setup

All experiments used PyTorch on CPU-only infrastructure:

- **Maximum steps per episode:** 2000
- **Training episodes:** 5000 per difficulty level

- **Hardware:** Intel Core i9 14900 CPU (no GPU); 64 GB RAM (not completely utilized)
- **Training time:** 4–6 hours per algorithm per level (approx)

4.3 Hyperparameter Configuration

Table 2: VPG + State Stacking

Parameter	Value
Actor LR	3×10^{-4}
Critic LR	10^{-3}
γ	0.99
Stack size	4
Hidden	[128,128]
Adv norm	Yes
Grad clip	1.0

Table 3: PPO + GAE

Parameter	Value
Optimizer	Adam
LR	3×10^{-4}
γ	0.99
GAE λ	0.95
Clip ϵ	0.2
Value coef	0.5
Entropy	0.01
Grad norm	0.5
Steps/update	2048
Epochs	4
Batch	64
Hidden	[128,128]

Table 4: Noisy DDQN + LSTM

Parameter	Value
Optimizer	Adam
Learning rate	10^{-3}
γ	0.99
Batch size	256
Replay buffer	100k
Warmup steps	2,000
Target sync	2,000
LSTM hidden	64
Noisy σ_{init}	0.017
Grad clipping	5.0

4.4 State Representation

- **VPG + State Stacking:** Concatenation of current and previous 3 observations (72-dimensional), providing explicit temporal context without recurrence. The frame buffer is initialised with zero-padding at episode start.
- **PPO + GAE:** Raw 18-bit observations processed by shared encoder with separate actor-critic heads.
- **Noisy DDQN + LSTM:** Raw 18-bit observations processed sequentially by LSTM, maintaining hidden state for implicit velocity estimation.

4.5 Exploration Strategy

- **VPG and PPO:** Stochastic policy exploration with entropy regularization.
- **Noisy DDQN:** Parameter space noise via Noisy Linear layers.

4.6 Curriculum and Multi-Task Learning

We experimented with **curriculum learning** (Level 1 $\rightarrow$ 2 $\rightarrow$ 3) for some variants. However, our best models were trained **directly on Level 3**, as direct training produced more robust policies for architectures with memory mechanisms.

4.7 Local Evaluation Strategy

- **VPG and PPO:** Model selection based on 100-episode (VPG) or 50-episode (PPO) moving average of training return. Best weights kept in memory and saved once at training completion.
- **Noisy DDQN + LSTM:** Evaluation every 50 episodes using 10 rollouts with fresh seeds. Best model saved based on evaluation performance.
- **Wall Obstacles:** Models trained and evaluated with `--wall_obstacles` flag.

4.8 Avoiding Overfitting

- **Gradient clipping:** Max norm 1.0 (VPG), 0.5 (PPO), 5.0 (Noisy DDQN)
- **Entropy regularization:** 0.01 for PPO; implicit via stochastic sampling for VPG
- **Advantage normalization:** Per-episode normalization in VPG prevents large gradient steps

5 Results

5.1 Development Phase Performance

Table 5 summarizes performance across phases.

Table 5: Development Phase Results (Mean Cumulative Reward)

Model	**Level 1**	**Level 2**	**Level 3**
Random	−1983.80	−1987.90	−1515.36
Heuristic	−198945.00	—	—
DDQN	—	−119975.50	−1519.66
D3QN	—	−115268.80	—
D3QN-PER	—	−194619.80	−1374.70
Noisy DDQN	—	−1981.20	−1519.36
Noisy D3QN	—	−1981.40	−1518.92
REINFORCE	—	−329918.60	—
A2C/A3C	—	−394476.00	—
PPO	—	−46891.80	—
DYNA-Q	—	−1988.40	—
Q-Learning + Clustering	—	−115268.80	—
Braitenberg Vehicle	—	−118721.40	−215369.84

Value-based methods consistently outperformed policy gradients in Phase 2, with Noisy networks providing marginal gains.

5.2 Final Test Phase Results

Table 6 presents Test Phase results.

Table 6: Test Phase Results (Mean Cumulative Reward)

Model	**Score**
VPG + State Stacking (Sub 26), GAE (Sub 29), Noisy DDQN + LSTM (Sub 14)	−1827.71
Modified TD3 (Sub 43), Modified GAE (Sub 41), TD3 (Sub 30), REINFORCE (Sub 23)	−1827.81
DDQN + State Stacking (Sub 17)	−1827.93
SAC (Sub 32)	−1828.10
Modified A2C (Sub 40)	−1828.15
REINFORCE + Eligibility Traces (Sub 48), Modified SAC (Sub 44), VPG + LSTM (Sub 39), REINFORCE + LSTM (Sub 38), Noisy D3QN + LSTM (Sub 16), D3QN + LSTM (Sub 12), DDPG (Sub 31), A2C (Sub 27), REINFORCE + State Stacking (Sub 25), VPG (Sub 24), Noisy DDQN (Sub 8)	−1828.41
VPG + LSTM + Eligibility Traces (Sub 49), D3QN-PER + LSTM (Sub 13), PPO (Sub 33), Noisy D3QN + State Stacking (Sub 21), Noisy D3QN (Sub 9), Rotate (Sub 2)	−1828.51

5.3 Learning Curves

Figure 2 shows training dynamics for best-performing models.

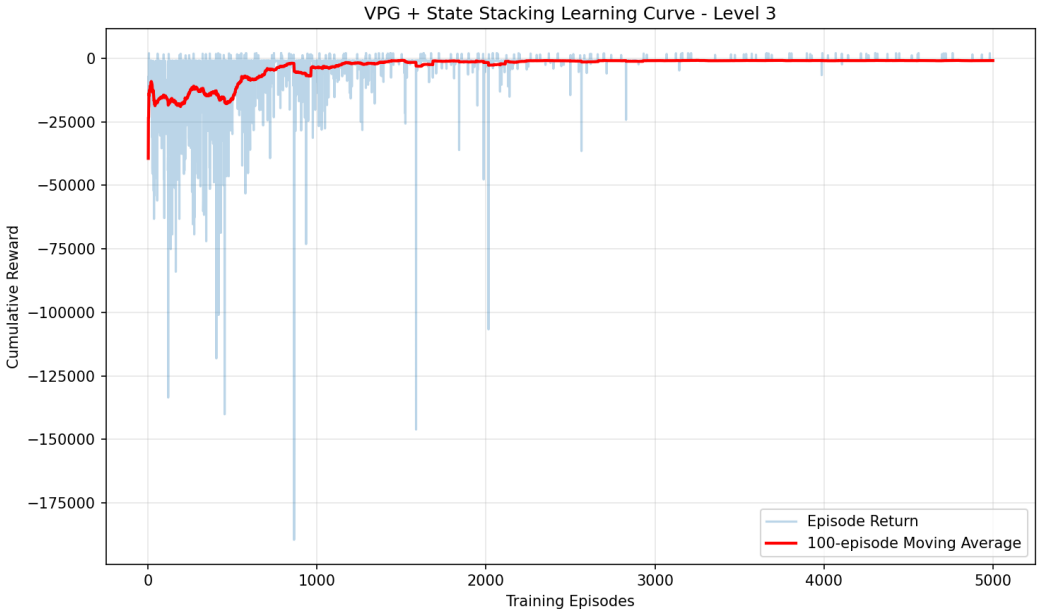


Figure 2: Learning curves for VPG + State Stacking, GAE, and Noisy DDQN + LSTM on Level 3.

5.4 Best Model Analysis

Three models achieved equivalent top performance (≈ -1828): **VPG + State Stacking**, **GAE**, and **Noisy DDQN + LSTM**.

Why VPG + State Stacking: Among the top-performing submissions, VPG + State Stacking is the preferred final model for three reasons. First, it achieves the joint-best score of -1827.71 without requiring a replay buffer, target network, or recurrent architecture. Second, the learned value baseline reduces gradient variance while remaining straightforwardly interpretable. Third, four-frame concatenation provides sufficient temporal context for Level 3 dynamics at a fraction of the implementation complexity of LSTM variants, and avoids the hidden-state initialisation and BPTT issues inherent in recurrent approaches.

Success Factors: All three top performers incorporated explicit temporal memory mechanisms (state stacking or LSTM recurrence), employed stable exploration strategies (stochastic policy or Noisy Networks), and demonstrated strong generalization from Level 3 training to easier difficulty levels. The equivalence in scores confirms that temporal integration is the dominant factor, not the specific algorithmic family.

6 Error Analysis and Discussion

6.1 Common Failure Modes

- **Level 1 (Static):** Oscillation without attachment (IR requires sustained contact), boundary trapping (box-corner-wall sandwich), and exploration collapse (high initial noise).
- **Level 2 (Blinking):** Lost track after disappearance (feedforward agents), premature detachment when box blinks, and timing errors approaching disappearing targets.

- **Level 3 (Moving):** Reactive chasing (perpetual lag behind box), interception failure (discrete angles limit precision), and velocity misestimation during blinking.

With Walls: Gap detection failures (limited sonar resolution), box-wall sandwiching, and backtracking (rotation without forward progress).

6.2 Limitations

The current approach has several key limitations: discrete actions with 22.5° and 45° granularity restrict precision, there is no explicit motion modeling beyond implicit dynamics via state stacking, and behavior-based decomposition was attempted but did not outperform the monolithic baseline.

6.3 Key Insights

- **Memory mechanism equivalence:** State stacking matched LSTM performance, suggesting explicit concatenation is as effective as learned recurrence for low-dimensional observations.
- **Policy gradient resurgence:** Temporal context from stacking reduced variance, enabling VPG/GAE to match value-based performance despite failing in Phase 2.
- **Architecture vs. algorithm:** Temporal integration mattered more than value estimation refinements—the gap between memory-equipped and feedforward agents exceeded differences between DQN variants.

7 Conclusion

7.1 Key Achievements

VPG + State Stacking achieved -1827.71, matching the best submissions across all algorithmic families with significantly lower implementation complexity than recurrent or value-based alternatives. Four-frame concatenation proved a computationally efficient substitute for LSTM recurrence, and 81 submissions provided a systematic comparison across value-based, policy gradient, actor-critic, and model-based families.

7.2 Main Takeaway

Temporal integration dominates algorithmic choice in partially observable robotics. The performance gap between memory-equipped and feedforward agents exceeded differences between RL algorithm families. For sensor-based POMDPs, architectural solutions for temporal reasoning matter more than algorithm-level refinements.

7.3 Future Directions

The most direct extensions are a systematic ablation of stacking size (our $k = 4$ was arbitrarily chosen) and explicit belief state estimation via particle filters to better handle blinking and moving box dynamics. Longer term, sim-to-real transfer and more realistic pushing physics remain open challenges.

LLM Usage Declaration

- Which Large Language Model (LLM) did you use?
  Kimi K2.5

- In which part did you use the LLM?

Part	**Used LLM?**	**Purpose**
Understanding concepts	Yes	Clarifying POMDP concepts, Noisy Networks mechanics, and GAE implementation details from course materials
Ideation / brainstorming	Yes	Discussing experimental strategies and report organization
Debugging code	No	All algorithm implementations were based on course pseudo-code and personal implementation
Plotting / visualization	Yes	Adding CSV logging and matplotlib visualization code to existing training scripts (ancillary plotting functions)
Grammar / writing style	Yes	Editing report sections for grammar (LaTeX formatting assistance)
Report structure	No	Already Given

- Did you refer to other sources?
  Yes. Primary sources included:
  - Course lecture slides (CS780: Deep Reinforcement Learning) for algorithm pseudo-code
  - "Reinforcement Learning: An Introduction" by Sutton & Barto (2nd Ed.) for foundational concepts
  - "Automatic Programming of Behaviour-based Robots using Reinforcement Learning" by Mahadevan & Connell (1991/1992) for environment background
  - PyTorch documentation for API reference

- Apart from any LLM usage, what was your own contribution?
  - **Implementation:** I personally implemented all core RL algorithms (DDQN, D3QN, PER, Noisy Networks, LSTM variants, VPG, PPO, GAE, etc.) based on course lectures and textbook descriptions. I conducted all 81 experiments, performed hyperparameter tuning manually, and analyzed results to draw conclusions.
  - **Analysis:** All performance comparisons, failure mode analysis, and insights about temporal memory mechanisms represent my own understanding from running experiments.
  - **Code:** The main training loops, network architectures, and agent policies were written by me; LLM assistance was limited to adding plotting/logging wrappers around existing code.
  - **Report Content:** The experimental results, submission history, and technical descriptions of my implementations are based on my actual work; LLM assisted only with LaTeX formatting and language polishing.

References

[1] S. Mahadevan and J. Connell, "Automatic programming of behavior-based robots using reinforcement learning," *Artificial Intelligence*, vol. 55, no. 2-3, pp. 311–365, 1992.

[2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction.* MIT Press, 2nd ed., 2018.